

The Converge System — Compile Intent, *Prove Every Step.*

The canonical blueprint for compiling a fuzzy brief — or a raw idea — into an autonomous, eval-gated system — **eight passes, one optional on-ramp, and one spine** to a closed loop. There is no architectural fork: Consensus sharpens the plans and hands them to one six-tier Task-Spec engine. A referee CLI — **cvg** — exposes the gates as stable machine contracts. v5 explains both sides: what every stage means, and exactly how an agent discovers, runs, interprets, repairs, and hands off each one.

8

PASSES ·
ONE PATH

13

CLAUDE CODE
SKILLS

6

TASK SIZES
XS → XXL

cvg

REFEREE CLI
V0.13.0

DOCUMENT

Canonical process blueprint · v5 (agent-operable)

CONVERGED MEANS

an eval passed — not that you feel done

The invariant — every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception: it does not lower, it closes. v5 records the one-spine method, Task-Spec 3.5.0, and cvg 0.13.0: self-describing commands, stable tokens, typed exit codes, JSON envelopes, dry runs, guided tracker setup, registration receipts, and explicit evidence boundaries. No pass is closed until its artifact, gate, proof, and lesson agree.

Introduction

Why Converge exists

The old loop ran in your head. A requirement arrived fuzzy, and you held the definition of "done" as private judgment — refined by taste, defended in review, never written down. That loop does not scale to AI engines, because an engine cannot execute a standard it cannot read. The bottleneck was never typing speed; it was that intent lived in a human and had to be re-supervised at every step. The AI-native engineer inverts this: take a fuzzy requirement and lower it through a repeatable descent — each step more precise than the last — until "done" is a machine-checkable condition an engine can satisfy unsupervised.

The main idea — compile intent, don't write the system

Stop writing the system. Compile it. Converge treats building as a **compilation pipeline** that lowers altitude in fixed stages: intent → structure → plan → tasks → code → loop. Each stage is a transformation with a typed output and a gate that must pass before the next stage runs — exactly how a compiler lowers source through intermediate representations down to machine code. You author the high-level intent and the checks; the pipeline emits the system. Your role shifts from author to compiler — you specify the descent and verify each lowering, rather than hand-writing the final artifact. When no client brief exists, an optional on-ramp — **Pass o • Capture** — compiles the raw idea into the BRD that starts the descent.

The thesis — build the factory from day one

You do not transform a workflow into a factory later. That retrofit — bolting evals, harnesses, and loops onto human-centric code never built to be gated — is the slow, expensive path, and it is the path almost everyone takes. Converge refuses it. The framework is factory-shaped from Pass 1: every pass already ends in a gate, every task is born with a runnable eval, and the harness is a control plane from the first file. So when you decide to step out, there is nothing to retrofit and nothing to rewrite; the gates already hold without you, and you walk out one gate at a time. And the factory teaches as it builds: understanding is a deliverable equal to working code, so every canonized pass closes with a lesson.

EVERY PASS

Lowers altitude

intent → structure → plan → tasks → code → loop. Each step closer to something a machine runs.

BINDS

An engine

the right tool for the altitude — Cowork to think, Claude Code to build, an adversary to attack, a fleet to crank.

ENDS AT

A gate

converged means an eval passed — not that you feel done. The gate is the unit of trust.

THE INVARIANT

Every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception — it does not lower, it **closes**. You are converged when the eval passes — not when you feel done. And in the operating model, no pass is closed until its lesson exists.

Contents

—	Introduction	<i>What Converse is, and why</i>
0	The Whole Method, On One Line	<i>the flat table + the invariant</i>
—	The Spine, Drawn	<i>eight passes, one on-ramp, one path</i>
0	Capture — idea-to-brd	<i>the optional on-ramp · altitude: idea</i>
1	Intent — brd-docs-to-tech-req	<i>altitude: intent</i>
2	Structure — tech-req-to-adrs	<i>altitude: system</i>
3	Decompose — reqs-to-swimlane-plans	<i>altitude: plan</i>
4	Consensus — sketch-plans-adversarial-review	<i>hands off to tasking</i>
—	One Spine, No Fork	<i>tasks all the way down · compatibility token explained</i>
5	Tasking — task-spec (six-tier engine)	<i>XS/S/M/L leaves · XL/XXL nodes</i>
①	Register — task-specs-to-issues	<i>the tracker as state</i>
6	Harness — stack-to-harness	<i>the quality moat</i>
8	The Loop — task-loop; Manager = future CI/CD	<i>altitude: runtime</i>
—	The Operating Model	<i>five beats per pass + the TEACH beat</i>
—	The Knowledge Home	<i>brain → docs → sketch → tasks → receipts</i>
—	The cvg CLI — a Referee at SOTA	<i>the command surface + --json + dod</i>
II	The Agent Operating the CLI	<i>discover → author → gate → repair → hand off</i>
—	Stage-by-Stage Agent Flows	<i>every pass · exact commands · exact boundaries</i>
—	Machine Contracts & Evidence Ledger	<i>JSON · exits · shipped vs proved vs next</i>
—	Why This Is Defensible	<i>two moats · positioning · honest boundary</i>
—	The Runtime, Proven	<i>what the skills already enforce</i>
—	One Method, Every Engineer	<i>the role lives in the eval</i>
—	The Method, End to End on One Feature	<i>the worked example</i>



The Whole Method, On One Line

Capture⁰ → Intent → Structure → Decompose → Consensus → Tasking
→ Register → Harness → Loop

A raw idea becomes a signed BRD, the BRD becomes a tech-spec, the tech-spec becomes ADRs, the ADRs become swimlane plans, the plans survive an adversarial review — and then, always, decompose into task-specs. There is **one spine and no decision branch**: a six-tier engine sizes every unit from a one-liner to a whole backbone, so there is no separate spec-driven paradigm to escape to. Each output is the next input; the chain descends from a fuzzy brief to a fleet running green — and every canonized pass is taught back to the owner before the descent continues.

#	PASS	SKILL	OUT · GATE
0	Capture (optional)	idea-to-brd	signed BRD — or a no-go record
1	Intent	brd-docs-to-tech-req	tech-spec · answers the brief
2	Structure	tech-req-to-adrs	docs/adrs/* + CONTEXT.md · system understood
3	Decompose	reqs-to-swimlane-plans	swimlane plans · plan-altitude only
4	Consensus	sketch-plans-adversarial-review	plans attacked, sharpened, provenance-gated · handoff to Tasking
5	Tasking	task-spec (six-tier)	XS/S/M/L leaves · XL/XXL nodes · each leaf has evals
①	Register	task-specs-to-issues	1 spec = 1 issue · the board is state
6	Harness	stack-to-harness	.claude/ + AGENTS.md · control plane
8	The Loop	task-loop --issue N	each task → green eval → PR

GUARDRAILS MAKE THE SEQUENCE CANONICAL

Pass 0 never touches the repo — idea in, brief out, and Pass 1 must not consume an unsigned brief. Pass 2 writes ADRs only (no code). Pass 3 is plan-altitude only (no tasks, no SQL). Pass 4 always hands off to Tasking. The current consensus schema still requires FORK: B as a compatibility stamp, but the method does not branch and agents do not choose a route. Pass 8 does not lower — it closes. No pass is closed until its artifact, gate, proof, and lesson agree.

The spine, drawn — eight passes, one on-ramp, one path

THE FULL DESCENT, NOW A STRAIGHT LINE

The whole descent is now linear. An optional Capture on-ramp feeds it when no brief exists; four passes lower altitude; at Consensus the plans are hardened and the path **always** hands off to Tasking — the six-tier Task-Spec engine absorbs the whole range a separate spec-driven paradigm used to cover. From Tasking the line runs on through Register, Harness, Execute, and the Loop. No branch, no rejoin — one spine, top to bottom.

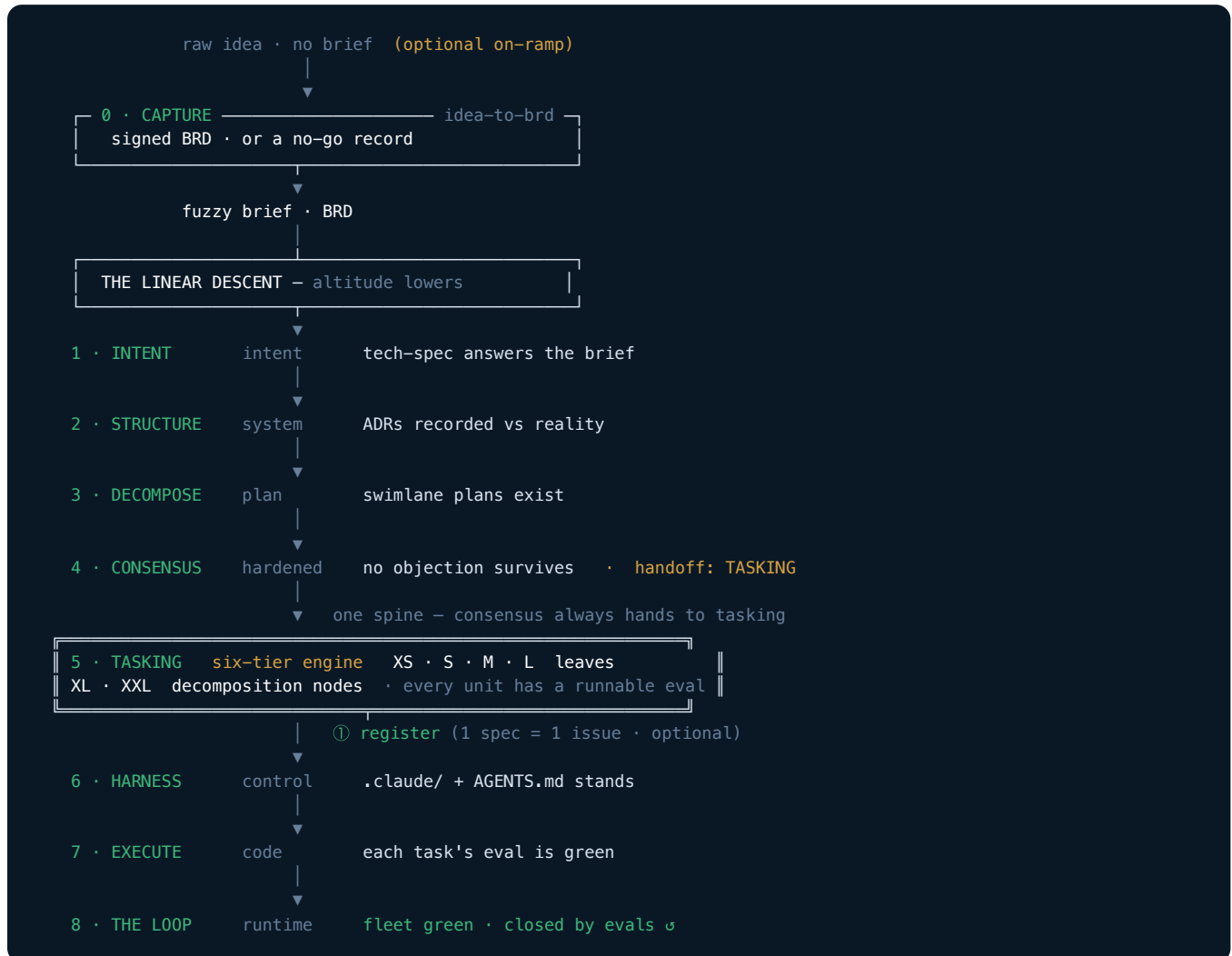


FIG 1 The 8-pass descent is a straight line — Consensus always hands off to Tasking, where the six-tier engine sizes each unit (XS/S/M/L leaves; XL/XXL nodes decompose further). Register is optional; Pass 8 closes the loop. Pass 0 is the optional on-ramp: raw idea in, signed brief out. One spine, top to bottom — no branch, no rejoin.

① No brief yet? Compile the idea into one.

PASS 0 OF 8 · OPTIONAL ON-RAMP · ALTITUDE: IDEA · *receive & interrogate the raw idea*

ENGINE · CLAUDE COWORK · SKILL · idea-to-brd v0.6.0 · OUT · SIGNED BRD OR NO-GO RECORD

Not every engagement starts with a client brief. Pass 0 takes a raw idea with no document behind it and compiles it into a BRD written in the owner's voice — or kills it cheaply. It scope-checks that this is one idea, grills the owner in **frontier rounds** — each round asks every question whose prerequisites are settled, numbered, with recommended defaults, and one reply answers the round — then drafts the brief and self-reviews it against the gate. Two honest exits: a signed BRD that Pass 1 can consume, or a no-go record parked with the reasoning intact.

SCOPE-CHECK

One idea

is this one idea? Explore the environment for facts first — facts never enter a round, and never block one.

GRILL

Frontier rounds

seven branches in leverage order — pain → do-nothing test → goal → scope boundary → success → constraints → pre-mortem.

DRAFT

BRD + self-review

docs/brd-<slug>.md from the template, in the owner's voice — then audited against the gate before sign-off.

GATE

The pain carries a **provenance-tagged number** — (measured) / (estimated) / (guessed) — with at least one KPI in the owner's terms; scope in/out are non-empty; every open question has a named owner; the do-nothing test and the pre-mortem ran; the Sign-off block is signed. The gate speaks in tokens — CHECK_BRD=PASS|FAIL|DRAFT_OK|NOGO_OK|... — and **canonical** must sit on the Sign-off verdict line: pending or draft there never authorizes the descent. Pass 1 must not consume an unsigned brief — and a no-go here is the **cheapest kill in the whole descent**.

*The frontier-rounds grill adapts Matt Pocock's **batch-grill-me**; Pass 0 adds the fixed branch map, the no-go exit, provenance tags, and the gate.*

1 The client hands you the problem. You produce the spec.

PASS 1 OF 8 · ALTITUDE: INTENT · *receive & comprehend the brief*

ENGINE · CLAUDE COWORK · SKILL · brd-docs-to-tech-req v0.6.0 · OUT · TECH-SPEC

Read the BRD like a senior engineer, not a stenographer. Find the real problem underneath the ask, name who actually hurts when it is unsolved, and pin the numbers that define success. Ambiguity left here compounds through every pass downstream — this is the cheapest place in the whole spine to kill it, and the most expensive to leave alone.

The interrogation runs in frontier rounds — facts never enter a round; unanswered questions ride twice, then become owned open questions. What survives becomes a typed **gap register** (GAP-001 · scope | definition | number | data · blocker | minor): a blocker left open fails the gate. Prior art is checked at the problem level only — never the codebase — and owners who are absent get an exported questionnaire, not silence.

UNDERSTAND

Read

the BRD like a senior engineer — the one-paragraph "solved" statement comes first: the real problem underneath, who hurts, the numbers that define success.

INTERROGATE

Grill

frontier rounds with recommended defaults; the 2–3 questions whose answers most change how this gets built; ambiguity here compounds downstream.

CRYSTALLIZE

Spec

replay the "Locked:" recap, then resolve into one technical answer — falsifiable requirements (must / should / could / won't), metrics traced to the BRD's KPIs.

GATE

You can restate the client's problem in one breath and show the tech-spec answers it — scope, data, and success metric named, every requirement falsifiable, every blocker gap closed. --draft validates structure but never authorizes; the canonical run passes only a spec whose Sign-off verdict line reads **canonical** — CHECK_Tech_SPEC=PASS is the only verdict that hands to Pass 2. Brief-in, spec-out — never blur them. And no premature technology: naming one here is the most common Pass 1 failure.

2 Greenfield or brownfield — and write the ADRs.

PASS 2 OF 8 · ALTITUDE: SYSTEM · *ground the spec, record the decisions*

ENGINE · CLAUDE CODE · ON THE REPO

SKILL · tech-req-to-adrs v0.4.0

OUT · docs/adrs/*.md + CONTEXT.md

Before any plan, name the ground. Brownfield is a system you did not write: open it end to end, and check the spec against the real schemas, sources, and jobs you find. Greenfield is a system that does not yet exist: confirm there is nothing to inherit, then ground the spec in the source systems and constraints it must honour. Either way you leave Pass 2 knowing the terrain — what is actually there, and what the work must respect.

Not every fact earns an ADR — the **worthiness test** keeps the record honest: a decision qualifies only if it is hard to reverse, stood on downstream, and could have been otherwise. Shared vocabulary goes to `docs/CONTEXT.md`, the glossary Pass 6 wires into the harness — terms only, pinned as they crystallize.

NAME THE GROUND

Field

brownfield (a system you didn't write) vs greenfield (nothing yet) — name it explicitly; the rest follows.

GROUND

Check

open the system end to end; reconcile the spec against real schemas, sources, and jobs.

RECORD

ADRs

write `docs/adrs/*.md` — grounding decisions that pass the worthiness test (facts + constraints), never how to build. That's Pass 3.

GATE

The system is understood and its grounding decisions are recorded durably, so Pass 3 can read them without re-deriving the terrain. An ADR records what is true — schemas, joins, constraints — never how to build. The moment an ADR says "build X", you've drifted into planning. An ADR says what is true; `CONTEXT.md` says what things mean.



Split the work along its natural seams.

PASS 3 OF 8 · ALTITUDE: PLAN · *decomposition · break it into swimlanes*

ENGINE · CLAUDE CODE · SAME SESSION

SKILL · reqs-to-swimlane-plans v0.7.0

IN · READS THE ADRs

Take the system understood in Pass 2 — same session, ADRs in hand — and split the work along its natural seams. Read the ADRs, then cut where the system already wants to be cut: by feature or by component. Each seam becomes one swimlane; each swimlane gets its own focus-oriented sketch plan. The common case is two lanes — transform versus serve, a medallion lane and a serving lane — but a system carrying several features splits into several lanes. The right number of lanes is the number of genuine seams. No more, no fewer.

Prefer the **highest seam, and the fewest** — cut at the published contract, above implementation detail. And before a lane earns a plan it passes the **deep-lane test**: you can say what it does, how you use it, and what it depends on — without reading its internals. A lane that can't be understood from its interface alone is a wrong boundary: move it or fold it.

DECOMPOSE

Seams

cut where the system is already jointed — a feature edge or a component edge, not an arbitrary slice.

SWIMLANE

One plan each

every seam becomes one swimlane carrying exactly one sketch plan — one lane, one plan, one focus.

GROUNDED

Reads ADRs

each lane inherits the architecture decisions in `docs/adr/` — it builds on Pass 2, not around it.

GATE

One sketch plan per swimlane, each listing features, dependencies, and a sane build order, inheriting the ADR decisions. Plan-altitude only — no atomic tasks, no code. The session carries from Pass 2; the understanding becomes the plans without leaving the session.

REFERENCE SHAPE — THE COMMON TRANSFORM → SERVE CASE

One concrete instance of the seams Pass 3 splits into: a transformation lane (medallion: raw → bronze → silver → gold) feeding a serving lane (MCP / API over gold). Other systems split by feature or component — this is the shape, not the rule.

4 Bring a different engine to refute the plans.

PASS 4 OF 8 · ALTITUDE: PLAN (HARDENED) · consensus · a different model attacks

ENGINE · CROSS-FAMILY ADVERSARY (codex / kimi) SKILL · sketch-plans-adversarial-review v0.5.0 CLI · cvg review

A single model won't refute itself hard enough. So Pass 4 brings a different engine to attack the plans, one at a time, as a skeptic who defaults to "refuted." It grounds each plan against the tech-spec and the ADRs — hunting contradictions, gaps, and silent assumptions — and every objection is either **FIXED** in a plan or **ACCEPTED** with a named owner. Nothing is silently dropped. Cross-model disagreement is not noise here; it is the entire point.

When argument can't settle an objection, settle it with a **throwaway prototype** — the smallest disposable artifact that answers the question, run against real data. The result decides **FIX** or **ACCEPT**; the prototype is **evidence, not deliverable** — throw it away.

ATTACK

Refute

a different model, one plan at a time, as a skeptic who didn't write it — default to refuted.

GROUND

Drift?

check each plan against the tech-spec and the ADRs — contradictions, gaps, silent assumptions.

SHARPEN

Fix / Own

every objection **FIXED** in a plan or **ACCEPTED** with a named owner — nothing silently dropped.

GATE — FALSIFIABLE, NOT HONOUR-SYSTEM

The gate reads a **stamped objection-log artifact**, never plan prose. It passes only when: a **different-family** adversary attacked (proven by a provenance stamp + input hashes computed by the referee — not a grep of a word); every objection is **FIX**-or-**ACCEPT**-with-owner; the plans have **not drifted** since the review (re-hashed); and the compatibility value **FORK: B** is present. The referee never edits — `cvg review --adversary codex,kimi` dispatches, the engine writes, `cvg review --check gates` → `CHECK_CONSENSUS=OK`. The compatibility stamp is fixed; there is no route decision: Consensus **always** hands off to Tasking.

One spine — no fork, no route decision.

V5 · TASKING IS THE ARCHITECTURE · RECURSION ABSORBS SCALE

The current Converge method does not fork at Consensus. A different-family adversary attacks the plans, every objection is settled, and the hardened plan tree goes directly to Tasking. Frontier engines execute well-scoped atoms reliably, and verified atoms compose back up more safely than one monolith. A single six-tier Task-Spec engine therefore carries the whole range — from a one-line change to a complete backbone — without a second specification paradigm.

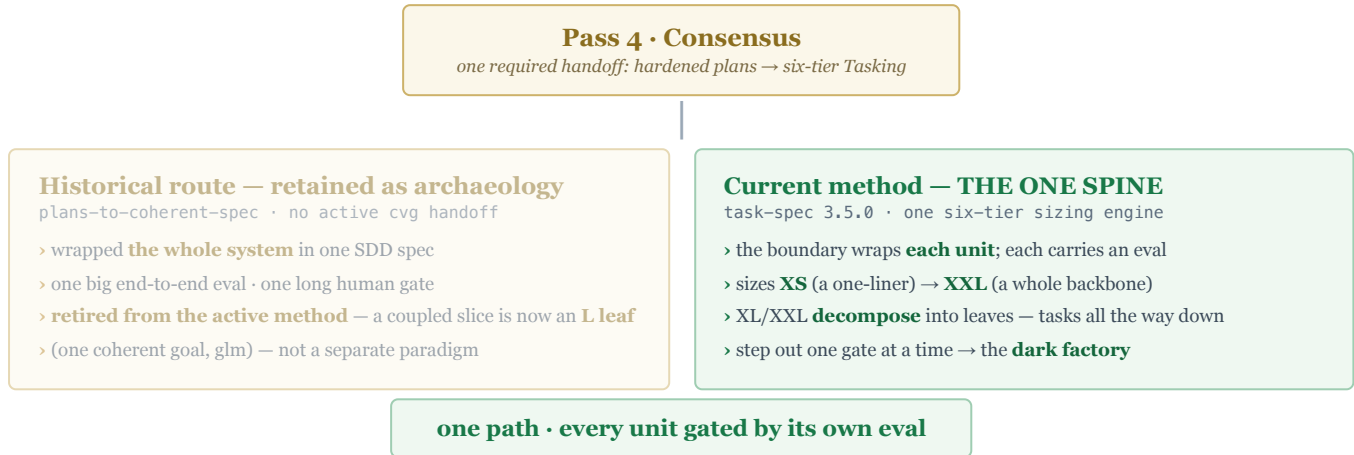


FIG 2 There is no architectural choice after Consensus. The current schema still stamps **FORK**: **B** for backward compatibility with the consensus gate; agents treat it as a required compatibility value, never as a prompt to choose a branch.

THE SIX-TIER ENGINE ABSORBS THE RANGE

Leaves run · nodes decompose

Leaves (XS/S/M/L) are runnable atoms — each fits one fresh context window and verifies as one PR / test-suite; a gate-checked **write-surface budget** ($XS \leq 1 \dots L \leq 5$ paths) catches a mis-sized unit. **Nodes** (XL/XXL) are not runnable — they **MUST** declare **children**: and expand into leaves. A worker dispatches the children, never the node — cvg **tasks gate** refuses a node.

WHY ONE PATH IS SAFER

Recursion, not escalation

Big work used to route **out** to SDD. Now it decomposes **in**: the same task-spec format, self-similar at every scale, so the seam contracts are written once and per-unit tasks can't drift. Research (clispec, Moai, Thread AI, AgentSpec — 2026) put the format at or above SOTA on every axis.

TASKS ALL THE WAY DOWN — AND BACK UP

The dark factory needs one unit model at every scale. A subjective slice becomes a human-checkpoint eval; a coupled slice becomes an L leaf; a whole backbone becomes an XXL node that decomposes to a tree of leaves. There is no route out: descend by decomposition, run only leaves, then compose verified results back up.

5 The six-tier engine — size every unit, from XS to XXL.

PASS 5 · THE SIZING MODEL · TASK-SPEC V3.5 · one engine, a one-liner to a whole backbone

ENGINE · TASK-SPEC (SIX-TIER)

SKILL · task-spec v3.5.0

GATE · cvg tasks validate

Every unit gets a t-shirt size — sized the agent-native way: not by human hours, by **context, verification, and blast radius** (the 2026 SOTA signal — Young, Anthropic, Vaughan, Vest). Two kinds. **Leaves** (XS/S/M/L) are runnable atoms that fit one fresh context window and verify as one PR / test-suite. **Nodes** (XL/XXL) are decomposition directives — not runnable; they expand into leaves. The write-surface budget is gate-checked, so a mis-sized leaf is caught before it ships and a node with no children is refused.

SIZE	KIND	WRITE BUDGET · BACKEND	EXAMPLE
XS	leaf	≤ 1 path · Kimi	a config key; bump a dependency
S	leaf	≤ 2 paths · Kimi	a /health endpoint + its test
M	leaf	≤ 3 paths · Kimi	a dbt model + schema + test
L	leaf · ceiling	≤ 5 paths · glm required	a service slice as ONE coherent goal
XL	node · decomposes	children ≥ 2 (owns none)	a whole swimlane → its leg leaves
XXL	node · decomposes	children ≥ 3 (owns none)	a whole backbone → swimlane nodes

THE RECURSION — THIS IS THE WHOLE IDEA

XXL backbone → XL swimlanes → M/S/L leg atoms → XS units. A worker runs the **leaves** and composes their verified results back up the tree. A budget that gets breached means the decomposition was too coarse — the signal to split further. Big work decomposes **in** (recursion), it never routes **out** (escalation).

GATE — CVG TASKS VALIDATE

Sizing is enforced, not advisory: a leaf over its write-surface budget WARNS "mis-sized — split or reclassify UP"; an XL/XXL node with too few children FAILS ("must decompose — no route out"); L requires `execution_backend: glm` + one coherent done-condition. The format is vendor-neutral and self-verifying — the same file runs on Claude, Codex, or Kimi.

5 Tasking, in practice — a backbone into eval-backed units.

PASS 5 OF 8 · ALTITUDE: ATOMIC UNIT · *the plans decompose into task-specs*

ENGINE · TASK-SPEC SKILL · SKILL · task-spec v3.5.0 · CLI · cvg tasks validate | gate | dod

Consensus hands off here, always. The hardened swimlane plans decompose into **eval-backed task-specs** — the trust boundary wraps each unit, and verification is per-unit. Trust is earned one unit at a time: each eval that goes green is a gate you no longer hold by hand. This is the path to the Dark Factory — many small, automatable gates a fleet can close on a schedule. On the proving ground, the whole analytics backbone is already tasked: **9 legs** → **9 task-specs**, wired into a dependency DAG, every one `validate ok · gate DELEGATE · dod COMPLETE`.

v3.4 made the cut recursive; v3.5 adds the vendor-neutral `tracker_ref` receipt used by Register. Each unit is a **vertical slice** (a tracer bullet, not a horizontal layer) sized by the six-tier engine; behaviors (**B-1**, **B-2**...) trace to evals and back; and the seam contracts hardened at Pass 4 ride in as guardrails, so a per-unit task can't drift from the consensus.

SIZE

Six tiers

XS/S/M/L leaves run; XL/XXL nodes decompose. The write-surface budget catches a mis-sized unit at the gate.

BIND EVAL

Done = ?

a runnable eval defines done — the test passes, the query returns N, the probe reaches a served answer.

TRACE

DoD matrix

`cvg tasks dod` renders every behavior → its verifying eval → result; an untraced behavior FAILS the DoD.

GATE — CVG TASKS GATE (SAFE-TO-DELEGATE)

Every task carries a runnable eval — no eval, not a task yet. The delegation gate runs two stages — structural + sizing validation, then a broken-eval guard — and **refuses to delegate an XL/XXL node** (a worker dispatches its children). Sign-off is a key-optional HMAC seal, **tamper-evident** — it proves the evals were not weakened after sign-off. Self-contained and vendor-portable: Claude, Codex, Kimi, or manual.

① Each Task-Spec becomes one tracked issue.

REGISTER · the tracker as state

SKILL · task-specs-to-issues v0.4.0

github | linear | jira · fake tests

CLI · cvg register

Register projects every `signed_off: true` Task-Spec onto exactly one tracker issue — a disposable state shadow of the repo-owned work. The tracker is a **pluggable backend** behind five verbs: `preflight` · `upsert` · `link` · `list-ready` · `write-result`. Upsert is idempotent by the spec ID; each `depends_on` edge becomes one `blocked-by` relation; the task's Exit Check travels into the issue as its close condition. After a successful upsert, Register writes only a receipt — `tracker_ref: <tracker>:<issue>` — back into frontmatter. The issue-side marker remains the idempotency key, so CI can use `--no-stamp-refs` without losing convergence.

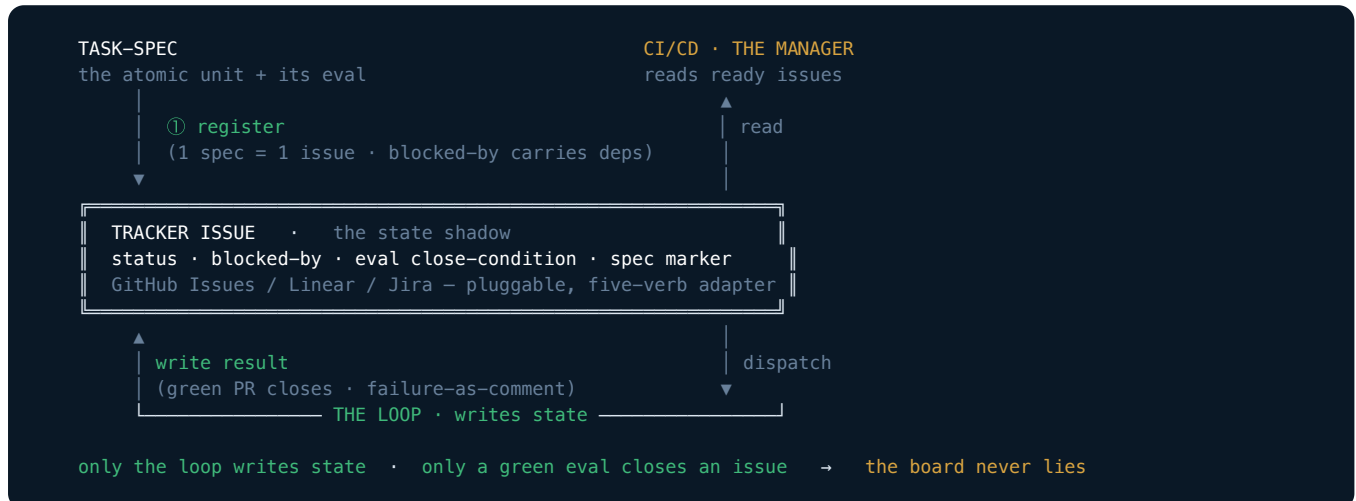


FIG 3 The repo owns the work; the tracker carries its dispatchable shadow. One Task-Spec registers as one issue, and a generic receipt points back to that issue without changing the work contract or its HMAC seal.

GUIDED SETUP · CVG 0.13.0

`cvg setup tracker linear` needs only `LINEAR_API_KEY`. It discovers visible teams, auto-selects one or asks for `--team KEY`, resolves the key to a UUID, and records the non-secret choice in `.cvg/config`. Credentials remain in the environment.

EVIDENCE BOUNDARY

The five-verb bridge is proven offline through the fake adapter. Live Linear connectivity exposed and fixed a malformed GraphQL payload. Full proving-ground registration remains a distinct closure gate.

THE SOURCE AND THE SHADOW

Stay at `task-spec` (repo files) when you want repo-local control — the specs travel with the code, version with the commit, need no external system. Add `task-specs-to-issues` when you want a board the loops read across sessions and machines. The tracker is replaceable shared memory; the repo files remain authoritative and can rebuild it.

6 Build the rails the work rides on — the quality moat.

PASS 6 OF 8 · ALTITUDE: CONTROL LAYER · harness · generate the build-system the stack needs

ENGINE · AGENTS-KBS-TECH-STACK · SKILL · stack-to-harness v0.3.0 · OUT · .claude/ + AGENTS.md

Pass 6 binds the harness to the system's tech stack — not to the tasks in flight. The dbt knowledge base, the MCP server agent, the DuckDB ruleset exist because the system runs dbt, MCP, and DuckDB — not because ISSUE-41 or ISSUE-42 named them. The tasks scope which stack components are in play, so you scaffold only what the system actually uses, never speculatively. The stack fills the harness; the tasks point at it. Tasking precedes Harness for exactly this reason: the work reveals which slice of the stack is in scope.

The control plane is verified, not hoped for: `quality-gate.sh --strict` exits 7 on a blocker. Cross-tool mirrors (`AGENTS.md`, Cursor, Copilot) never clobber hand-edits — a changed mirror ships as a `.proposed` sibling. The `CONTEXT.md` glossary is wired in, and **a manual step deserves a wizard, not prose** — interactive scripts under `.claude/scripts/`.

SCOPE

Read stack

the tasks reveal which techs are in play (dbt + duckdb + mcp) — scaffold exactly those, not speculatively.

SCAFFOLD

Agents + KBs

paired architect + developer per tech, grounded in real docs, plus KBs and rules.

EMIT

Cross-tool

`AGENTS.md` · Cursor · Copilot — every engine inherits one contract.

GATE

The control plane stands and is grounded in the system's tech stack — the tasks scope which components are in play, the stack fills the content. This is the quality moat: the files that make any commodity agent succeed, and the one artifact that outlives the project.

§ Build the execution loop. Schedule the Manager later.

PASS 8 · THE LOOP — *loop engineering*

Loop Engineering is the discipline of designing the loops that prompt agents, rather than prompting agents. The unit of work is no longer the message but the cycle. One primitive recurs everywhere — take an action, read the feedback the environment returns, decide the next move, repeat until the **goal condition** holds. Convergence is defined by the goal, never by attempt-count: a loop that stops because it tried three times has not converged; a loop that stops because the eval is green has.

Pass 8 · The Loop — task-loop (the execution loop). This is the loop you build. It takes ONE issue (`--issue N`, handed to it by a human or by CI — you never pick the task), then **READs** its task-spec plus the cited ADRs and the grounded harness, **ACTs** on a branch inside `touches_paths`, and runs the task's own **EVAL**. **RED**: the exact failure feeds back into a tight local loop, bounded by `budget_iterations`. **RED** twice with the same failure: stop patching, start diagnosing — reproduce minimally, hypothesize, verify, fix once. **GREEN**: **SETTLE** opens a Pull Request that closes the issue.

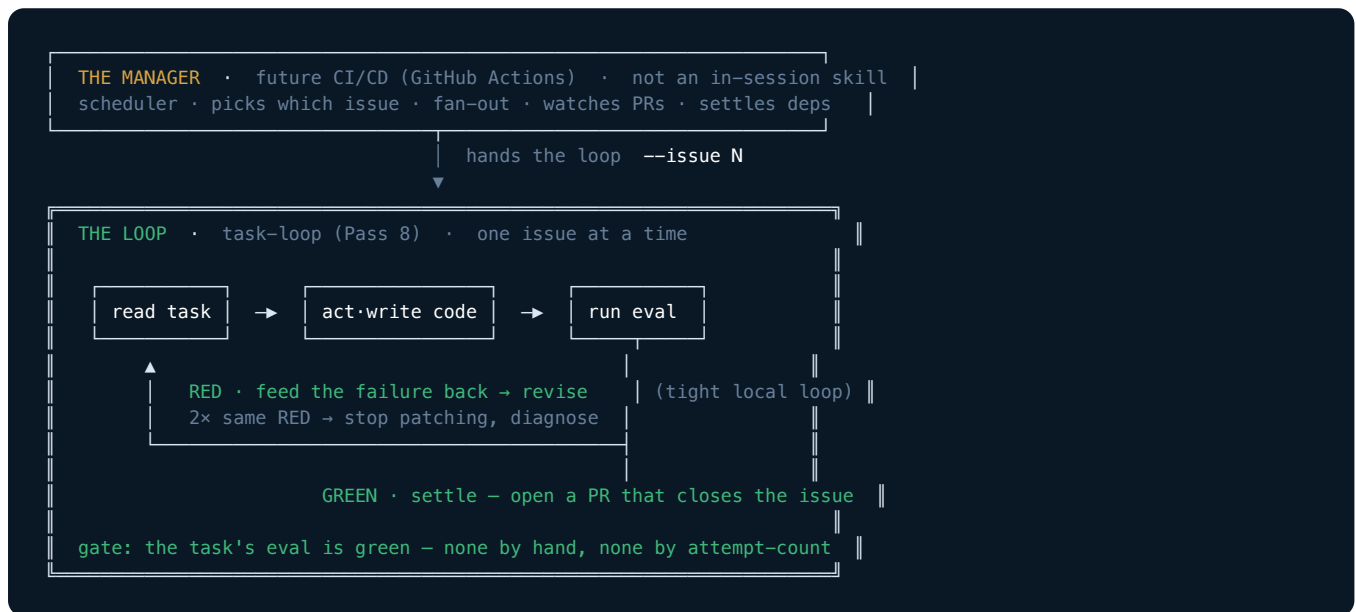


FIG 4 The execution loop — task-loop takes one issue, writes code, runs the task's eval, feeds RED back until GREEN, then opens a PR. The Manager that picks which issue and fans out is a future CI/CD concern (GitHub Actions), not an in-session skill.

GATE

Pass 8 — the task's eval is green; every issue closed by its OWN eval, none by hand, none by attempt-count. Schedule this loop in CI/CD and the human steps out: that is the Dark Factory, one issue at a time.

The operating model — five beats per pass.

HOW THE METHOD BUILDS ITSELF · TRACK R CURRENT · TRACK M PARKED

Converge is built with its own method. **Track R** validates the method one pass at a time, and each pass is a complete vertical slice of five beats — **in order, never bundled**. **Track M** — the execution machine, fixture-tested — waits at Milestone 0 and resumes at/around R8.

BEAT	WHAT HAPPENS · WHERE IT LANDS
1 · UNDERSTAND	read the pass's skill end to end, compare against the field, deliver a hardening list — analysis only, no edits
2 · RUN	execute the pass for real on the proving ground; the owner judges every output
3 · CANONIZE	iterate skill + artifact until the owner calls it canonical — blueprints are canonical vo ; a later pass may send a retro-edit back
4 · IMPLEMENT	the cvg subcommand for this pass — frame → dispatch → collect → gate, thin by design
5 · PROVE	re-run the pass through the CLI on the same input; artifacts and gate verdicts must match the golden manual run — nothing to trust, only to diff

THE TEACH BEAT — INSIDE THE C-BEAT

Closing CANONIZE includes the TEACH beat: run `pass-to-lesson` on the just-canonized artifacts — every component taught: what it is, why it is shaped this way, the decision it encodes, what breaks downstream without it — `check-lesson.sh` green, the lesson committed under `docs/lessons/`. **A pass is not closed until its lesson exists.** Rule 0 is the stance: go slow and teach — the owner's understanding is a deliverable equal to working code. The companion explains decisions, never reopens them.

ROUNDS · WHERE THE METHOD STANDS

Rounds are named `R<pass>.<beat>` — `Ro.U ... R8.P`. Snapshot, 2026-07-23: **Passes 0 through 4 are closed end-to-end** on the proving ground. Tasking has cut all nine backbone legs into validating, delegable, DoD-complete Task-Specs. Register and guided setup are implemented in cvg `0.13.0`; the bridge is proven offline and Linear connectivity has been exercised live, while the full `R①` proving-ground closure remains distinct. Harness and Loop are the next end-to-end rounds.

The knowledge home — the cvg/ workspace.

FIVE FOLDERS, ONE LIFECYCLE EACH

Every consuming project gets one Converge home — `cvg/`, named for the CLI — with `INDEX.md` as the front door. Artifacts flow one way and never backward: **brain feeds** → **docs agree** → **sketch explores** → **tasks execute** → **receipts prove**.

FOLDER	KIND	LIFECYCLE
brain/	user-generated inputs	raw, append-only, never gated
docs/	consensus artifacts	gated, permanent, never deleted — BRD · tech-spec · adrs/ + <code>CONTEXT.md</code> · lessons/
sketch/	drafts	transient — sharpened at Pass 4, decomposed into tasks at Pass 5
tasks/	sealed execution units	HMAC-stamped, tracked
receipts/	evidence — gate verdicts, pass receipts	write-once

ONE HOME

Per project

the workspace is named for the method's CLI, not the tool of the moment; dot-space (`. cvg/`) stays reserved for machine config.

ONE DIRECTION

Never backward

inputs feed, agreements gate, drafts expire, units seal, evidence freezes — nothing flows upstream.

ONE TRUTH

The repo

second-brain tools pull from the repo, never the reverse — the repo stays the single source of truth.

THE CONVENTION

The repo stays the single source of truth; second-brain tools pull from it, never the reverse. `. cvg/` is reserved for machine config. `MAP.md` answers "where do I look for X" in ten lines, and `INDEX.md` is the workspace's front door.

The cvg CLI — a referee at 2026 SOTA.

REFEREE, NEVER A PLAYER · AGENT-NATIVE · V0.13.0

Every pass is gated by one CLI — **cvg** — wrapping the proven skill scripts. It holds **zero model credentials** and makes no LLM calls: it frames prompts, dispatches engine CLIs headlessly (codex / kimi), and gates their output. Wrapped gates are **byte-exact pass-throughs**; bash-3.2-safe, stdlib-only, dep-free in CI. Every gate ends in ONE stable, greppable token and exit codes are contracts — a harness never parses prose. It exposes the implemented command surface while keeping closure claims tied to the proving ground.

```
# the whole 8-pass chain — one name, one token each
cvg capture → CHECK_BRD=PASS      cvg decompose → CHECK_PLAN=OK
cvg intent  → CHECK_TECH_SPEC=PASS cvg review --adversary codex,kimi → REVIEW=OK
cvg structure → CHECK_ADR=OK      cvg review --check → CHECK_CONSENSUS=OK
cvg tasks validate | gate | accept | dod      cvg doctor → DOCTOR=OK

# agent-native SOTA — wrap ANY command in one uniform envelope
$ cvg --json capture
{ "ok":true, "command":"capture", "token":"CHECK_BRD=PASS",
  "exit_code":0, "changed":false, "error":null,
  "meta":{"schema_version":"1.0", "cvg_version":"0.13.0" } }

cvg agent-context → command tree + exit-code taxonomy (retryable), as JSON
cvg setup tracker linear → discover team · record choice, never key
cvg register --check → validate 1:1 mapping + dependency DAG
cvg <mutation> --dry-run → reports intent, changes nothing (changed:false)
```

FIG 5 The referee surface. Default output is byte-parity prose ending in a token; **--json** wraps any command in a uniform {ok, data, error, meta} envelope without touching the default path; **agent-context** is the machine-readable manifest. Measured at or above the 2026 agent-native-CLI SOTA.

AGENT-NATIVE · 2026 SOTA

--json · agent-context · dod

Benchmarked vs clispec.dev / cli-agent-spec + 3 more: a uniform **--json** envelope on every command, an exit-code taxonomy with **retryable/side_effects**, a self-describing manifest, **--dry-run** on mutations, and the DoD/traceability view — all shipped.

PROVING GROUND · TRACK R

tests/uc-analytics

greenfield e-commerce Postgres, seed 42. Passes 0→4 are closed: signed BRD / tech-spec / ADRs / swimlanes, plus a real cross-family adversary sharpened the seams to **CHECK_CONSENSUS=OK**. Tasking has 9 specs, all gate **DELEGATE · dod COMPLETE**; later round closure is labeled separately.

DOGFOODING — THE MACHINE IS BUILT BY THE METHOD

Every **cvg** subcommand reproduces the canonical manual run — artifacts and gate verdicts diffed against the golden reference — before it ships. Gates ship as versioned exit contracts whose regression suites prove the negatives fail: a fixture that wrongly **PASSes** on the previous version is caught, because evals must discriminate. The result is a referee you can point an autonomous engine at — it never bluffs a pass.

The Agent Operating *cvg*.

The first half explains the method. This half explains the machine behavior: how an autonomous agent discovers the available surface, creates the right artifact at the right altitude, invokes the referee, interprets a stable verdict, repairs only the owning stage, and advances without inventing state.

DISCOVER agent-context → **AUTHOR** with the stage skill → **GATE** through cvg → **INTERPRET** token + exit → **REPAIR OR HAND OFF**

One protocol, repeated at every altitude. The CLI is the referee; the skills and engine sessions are the players. The agent never mistakes validation for authorship.

The universal agent protocol.

THE SAME CONTROL LOOP AT EVERY PASS · ONLY THE ARTIFACT AND GATE CHANGE

An agent does not "run Converge" as one giant prompt. It advances a typed state machine one gate at a time. Each pass owns one transformation; cvg publishes the evidence contract that says whether the transformation is allowed to hand off.

1	Discover	Call <code>cvg agent-context</code> once. Learn command names, tokens, mutability, retryability, and the current CLI version without scraping help prose.
2	Locate state	Read <code>cvg/INDEX.md</code> , then the artifact owned by the current pass. Never infer progress from a branch name, tracker column, or conversation memory.
3	Author	Use the stage skill to create or sharpen the artifact. The CLI usually validates; it does not replace the authoring skill or stakeholder decision.
4	Preview	For a mutation, prefer <code>--dry-run</code> or the stage's read-only check. Confirm targets, blast radius, credentials boundary, and expected receipt before changing state.
5	Gate	Invoke <code>cvg ... --json</code> . Trust the exit code plus stable token; do not infer success from a pleasant sentence, an engine exit 0, or an artifact merely existing.
6	Settle	GREEN → persist proof and hand the typed output to the next pass. RED → repair the artifact in the pass that owns it, rerun the same gate, and never patch downstream around an upstream gap.

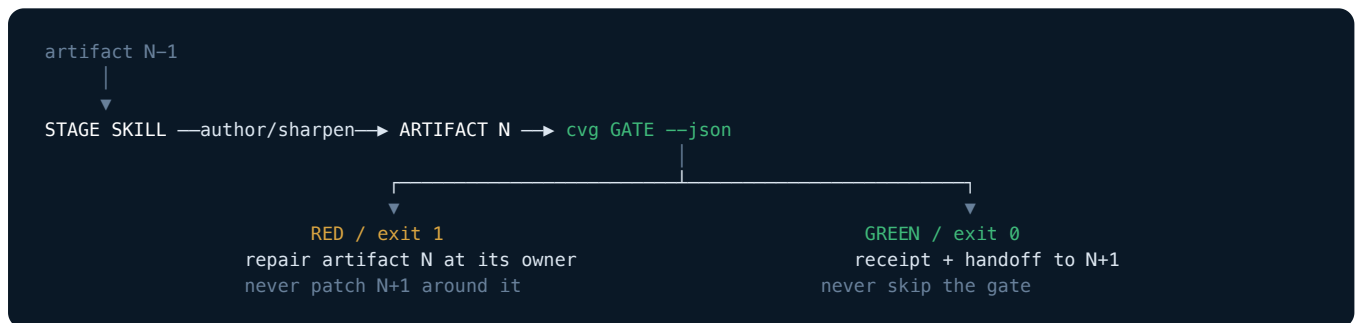


FIG 6 The agentic control loop. The stable part is the protocol; the changing part is the artifact type, authoring skill, gate token, and handoff.

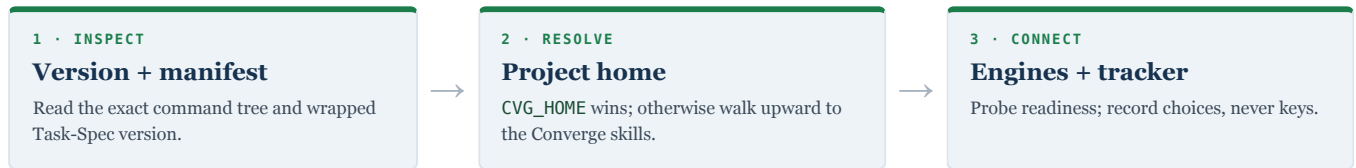
THE MOST IMPORTANT DISTINCTION

A skill transforms; cvg adjudicates. `cvg capture` does not interview the owner, and `cvg intent` does not write the tech-spec. An agent that asks the referee to be the player has removed the very separation that makes the evidence trustworthy.

Before Pass 0 — bootstrap the agent.

SELF-DESCRIPTION · PATH RESOLUTION · CONNECTIVITY · SECRETS BOUNDARY

The agent begins by establishing what exists, where it is, and which external surfaces are reachable. This is not a pass: it creates no product artifact and lowers no altitude. It prevents an agent from guessing commands, silently omitting unavailable engines, or storing credentials in project state.



```

# machine discovery — one call, no prose scraping
$ cvg version
cvg 0.13.0 (task-spec 3.5.0)
$ cvg agent-context > /tmp/cvg-manifest.json

# readiness board — read-only
$ cvg setup --json
{ "token":"SETUP=READY|INCOMPLETE", "changed":false, ... }

# guided Linear binding — records only the non-secret choice
$ export LINEAR_API_KEY=...
$ cvg setup tracker linear [--team CVG]
SETUP_TRACKER=OK
# .cvg/config stores tracker=linear + team UUID; never the key
  
```

QUESTION	AGENT RULE
Can I mutate?	Read the manifest's mutating flag; use <code>--dry-run</code> when supported; require the authority appropriate to the target.
Can I retry?	Exit 20/21/22 are dispatch availability/timeout/engine errors and are retryable; exit 1 is an artifact gate failure and must be repaired, not blindly retried.
Where are secrets?	In the environment or secrets manager. <code>.cvg/config</code> records a backend/team choice only. If the CLI cannot read the key from the environment, it cannot leak a stored copy.
What is authoritative?	Repo artifacts and their gates. Tracker and indices are projections; conversation memory is orientation, never proof.

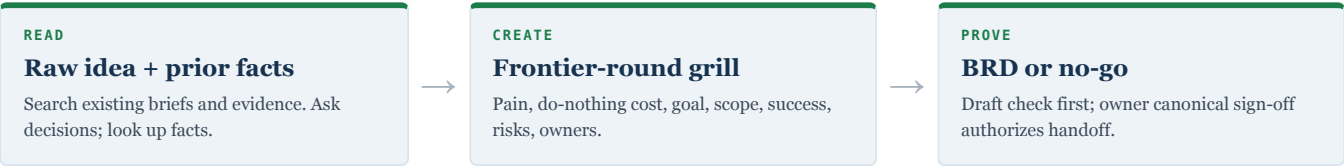
BOOTSTRAP EXIT

The agent can name the CLI and Task-Spec versions, locate the Converge home, identify every available command and its token, and distinguish unavailable infrastructure from a red artifact. Only then does it enter the first applicable pass.

① Agent flow · Capture.

OPTIONAL ON-RAMP · IDEA-TO-BRD 0.6.0 · CHECK_BRD

Enter only when no usable BRD exists. The agent works in the owner's language and stops before requirements, solution shape, or technology. This pass may legitimately end the project with a durable no-go.



```
# while authoring – structure only, never authorization
cvg capture --draft cvg/docs/brd-revenue.md --json
CHECK_BRD=DRAFT_OK|DRAFT_INCOMPLETE

# after the owner signs canonical + ISO date
cvg capture cvg/docs/brd-revenue.md --json
CHECK_BRD=PASS

# honest early exit
cvg capture --no-go cvg/docs/no-go-revenue.md --json
CHECK_BRD=NOGO_OK
```

CONTRACT	AGENT BEHAVIOR
Input	One raw idea; no existing client brief. If the idea contains independent systems, choose one slice before interviewing.
Output	docs/brd-<slug>.md in the owner's voice, or docs/no-go-<slug>.md stating why and what would reopen it.
On red	Repair the BRD: provenance-tag numbers, owned questions, explicit in/out scope, canonical sign-off. Do not start writing requirements.
On green	Persist the gate receipt; optionally teach the brief; hand the exact signed artifact to Pass 1. A no-go has no downstream handoff.

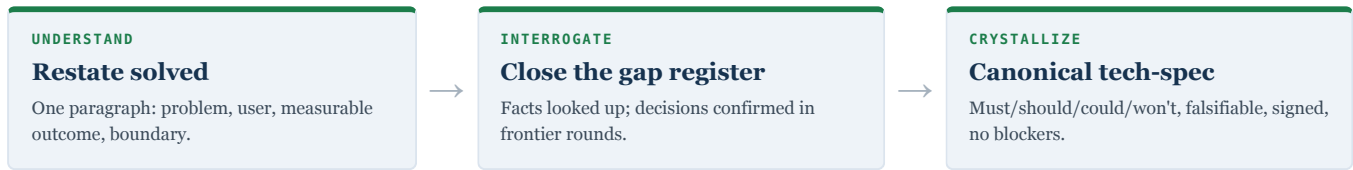
DO NOT CONFUSE

cvg capture validates an artifact that already exists. The idea-to-brd skill performs the interview and drafting. A green draft token is not owner authorization.

1 Agent flow · Intent.

BRD → FALSIFIABLE TECHNICAL CONTRACT · BRD-DOCS-TO-TECH-REQ 0.6.0

Pass 1 translates an owner-approved problem into engineering intent. The agent separates environment facts from owner decisions, asks only the questions whose answers alter the build, and produces one canonical tech-spec whose requirements and metrics trace back to the brief.



```

# structural iteration
cvg intent --draft cvg/docs/tech-spec-revenue.md --json
CHECK_TECH_SPEC=PASS|FAIL # draft semantics remain non-authorizing

# canonical exit
cvg intent cvg/docs/tech-spec-revenue.md --json
CHECK_TECH_SPEC=PASS # canonical sign-off + no blocker gaps
  
```

CONTRACT	AGENT BEHAVIOR
Input	The exact signed BRD from Pass 0 or the client. An unsigned brief is an upstream failure, not permission to infer intent.
Output	A tech-spec with prioritized, falsifiable requirements; metrics traced to BRD KPIs; a typed gap register; confirmed decisions; explicit sign-off.
On red	Return to the failing requirement, metric, gap, or sign-off. Ask the owner only for the missing decision; never promote a preference or guess into fact.
On green	Freeze the technical intent and hand it to Pass 2. Technology choices that require repo evidence still belong to Structure, not here.

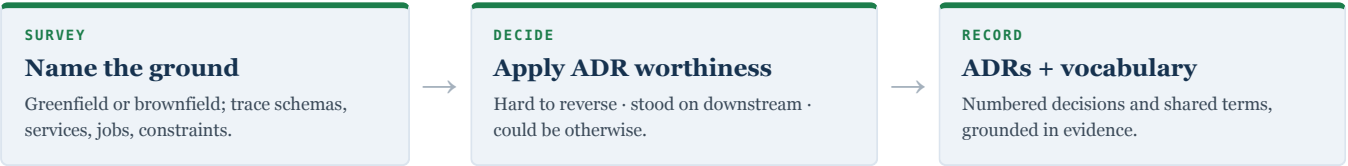
ALTITUDE FAILURE

A spec that already dictates tables, classes, or commands has fallen into planning. Pass 1 defines observable behavior and constraints; Pass 2 grounds decisions against the real system.

2 Agent flow · Structure.

GROUND THE SPEC IN THE LIVE SYSTEM · TECH-REQ-TO-ADRS 0.4.0

The agent now opens the repo end to end. It distinguishes facts from decisions, writes only ADR-worthy choices, and pins shared language in `CONTEXT.md`. The output explains what is true and why; it does not yet plan the implementation.



```
# repeat while proposed ADRs remain
cvg structure --dir cvg/docs/adrs --json
CHECK_ADR=OK|FAIL

# final handoff requires no proposed decision left
cvg structure --final --dir cvg/docs/adrs --json
CHECK_ADR=OK
```

CONTRACT	AGENT BEHAVIOR
Input	The canonical tech-spec plus the live checkout. Read the system; do not reconstruct terrain from stale documentation alone.
Output	docs/adrs/NNNN-*.md for load-bearing decisions and docs/CONTEXT.md for domain terms.
On red	Repair missing evidence, status, numbering, decision/alternative/rationale, or unresolved proposal. Do not solve it by adding plan steps to an ADR.
On green	The terrain is durable enough for decomposition. Hand the tech-spec, ADR set, and vocabulary together to Pass 3.

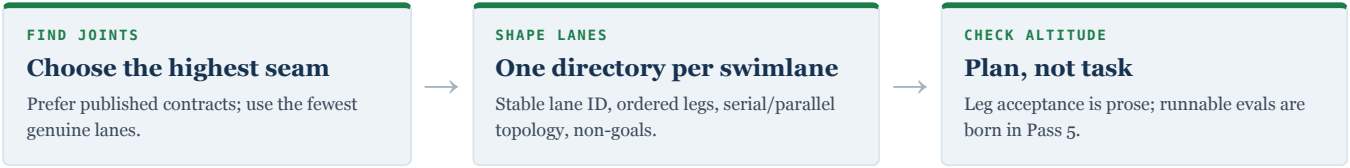
DO NOT CONFUSE

An ADR says what was decided against the ground. It does not say "build file X, then run Y." That is Pass 3 plan altitude and later Tasking.

3 Agent flow · Decompose.

SYSTEM DECISIONS → SEAM / SWIMLANE / LEG TREE · 0.7.0

The agent cuts the system where independently finishable work actually exists. Each seam owns one swimlane directory; each lane contains a lean plan plus leg files. Plans define interfaces, dependencies, non-goals, and acceptance intent — never tasks, code, or executable SQL.



```
# gate the whole tree, not one pretty plan
cvg decompose --dir cvg/sketch --json
CHECK_PLAN=OK

# expected shape
cvg/sketch/swimlane-transform/
swimlane-transform.plan.md
swimlane-transform-leg-01-silver.md
swimlane-transform-leg-02-gold.md
```

CONTRACT	AGENT BEHAVIOR
Input	Canonical spec, ADR set, and vocabulary. The agent must be able to cite which decision grounds each lane boundary.
Output	A complete swimlane tree: one lean PRD per seam, one file per finishable leg, dependency topology, acceptance intent, and no orphan leg.
On red	Repair missing lane metadata, unstable IDs, leg gaps, orphans, absent non-goals, topology, or altitude leakage. Never hide a bad seam by adding more detail.
On green	Freeze the attack surface and hand every live plan file to a fresh, different-family Pass 4 adversary.

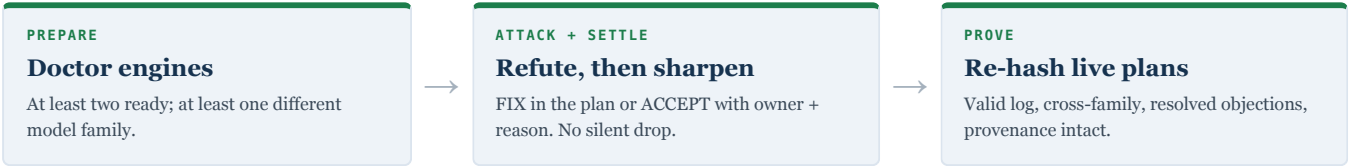
THE DEEP-LANE TEST

If a consumer cannot say what a lane does, how to use it, and what it depends on without reading its internals, the boundary is not deep enough. Move it or fold it before Consensus.

4 Agent flow · Consensus.

DIFFERENT-FAMILY ATTACK · PROVENANCE · SETTLEMENT · DIRECT HANDOFF TO TASKING

Pass 4 deliberately separates generation from judgment. cvg frames sealed inputs, dispatches read-only engine CLIs, captures raw evidence, normalizes a structured objection log, and stamps hashes. The authoring agent then settles every objection in the plans before the gate can pass.



```
cvg doctor --json
DOCTOR=OK

# mutating dispatch: preview if needed, then run
cvg review --adversary codex,kimi --dry-run --json
cvg review --adversary codex,kimi --json
REVIEW=OK

# after FIX / ACCEPT settlement in the plans + log
cvg review --check --json
CHECK_CONSENSUS=OK
```

VERDICT	AGENT BEHAVIOR
20 · SKIP	Requested engine is absent. Retry with an available cross-family engine; do not silently continue without independence.
21 · TIMEOUT	The mandatory wall-clock cap fired. Scope per swimlane or retry within policy; no artifact means no consensus.
22 · ENGINE_ERROR	Output was unparseable or invalid even if the provider exited 0. Preserve raw evidence, diagnose the adapter, fail closed.
Gate red	Repair open objections, missing owners/reasons, schema defects, or plan-hash drift. The referee never edits plans for the agent.

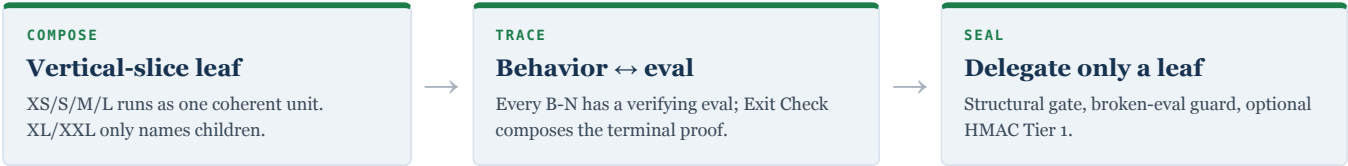
ONE-SPINE HANDOFF

The current schema still checks FORK: B as a compatibility value. The agent does not choose a route: a green Consensus always hands the hardened plan tree directly to six-tier Tasking.

5 Agent flow · Tasking.

HARDENED LEGS → SEALED, EVAL-BACKED TASK-SPECS · V3.5.0

Every leg becomes one or more Task-Spec leaves. The agent sizes by context, verification, and write surface; binds behaviors to runnable evals; declares tools, timeout, sandbox, retries, scope, and forbidden paths; then proves that the definition of done is complete before delegation.



```
cvg tasks validate cvg/tasks/T-revenue-gold.md --json
cvg tasks dod      cvg/tasks/T-revenue-gold.md --json
DOD=COMPLETE

# owner-authorized sign-off; nodes are refused
cvg tasks gate cvg/tasks/T-revenue-gold.md --stamp
VERDICT: DELEGATE · TIER=1

cvg lint --json
cvg ready --json
```

CONTRACT	AGENT BEHAVIOR
Input	All sharpened swimlane plans and legs, plus the decisions and seam contracts they cite.
Output	Task-Spec v3 leaves with behavior, contract, guardrails, operations, evals, retry policy, agent contract, dependencies, and optional signed-off seal.
On red	Split or resize excessive scope; add missing children to nodes; repair eval syntax; close behavior/eval trace gaps; remove ambiguous done conditions.
On green	The backlog is dispatchable. Register may stamp tracker_ref later; that receipt is metadata and never becomes the work's idempotency key.

NO EVAL, NOT A TASK YET

A well-written goal is not an execution contract. The agent cannot delegate until the task proves how another engine will know it is done, inside declared scope and bounded retries.

① Agent flow · Register.

REPO TRUTH → IDEMPOTENT TRACKER SHADOW · TASK-SPECS-TO-ISSUES 0.4.0

Registration is optional and sideways: it lowers no altitude. The agent first proves the repo graph, then connects the chosen adapter, upserts one issue per signed-off spec, mirrors dependencies, stamps receipts, and verifies that the board is a faithful projection.



```

export LINEAR_API_KEY=...
cvg setup tracker linear [--team CVG]
SETUP_TRACKER=OK

# rich read-only graph check before external writes
cvg register --check --tracker linear --json

# idempotent mutation; receipt write-back is explicit
cvg register --tracker linear --json
REGISTER=OK
cvg register --check --tracker linear --json
CHECK_REGISTER=OK
  
```

INVARIANT	WHAT THE AGENT VERIFIES
Source wins	The Task-Spec owns title, body, dependencies, and eval. Tracker edits never flow back into the work contract.
Idempotent	Re-running updates by the issue-side spec marker/external ID; it never creates a second issue for the same spec.
Receipt only	tracker_ref is a jump link. Use --no-stamp-refs in CI when the checkout must remain unchanged.
Honest status	Bridge shipped; fake-adapter registration proved offline; live setup exercised. Do not label R① closed until a real board passes the full mapping gate.

EXTERNAL-WRITE BOUNDARY

Registration creates or updates shared tracker state. Resolve the exact tracker/team, preview the graph, and obtain the authority required by the operating environment before the mutating call.

6 Agent flow · Harness.

TASKS REVEAL THE STACK · STACK-TO-HARNESS 0.3.0 · QUALITY MOAT

The agent derives the control plane from the in-scope Task-Specs — never from a wishlist. For every technology actually touched, it builds paired architecture/development roles, grounded knowledge, doctrine, rules, universal closers, and cross-tool mirrors so every execution engine inherits one contract.



```
# current v0.13.0 boundary: no `cvg harness` subcommand yet
# invoke the stage skill / its proven scripts directly
bash skills/agents-kbs-tech-stack/scripts/scaffold.sh --techs "$TECHS"
bash skills/agents-kbs-tech-stack/scripts/install-closers.sh
bash skills/agents-kbs-tech-stack/scripts/quality-gate.sh
bash skills/agents-kbs-tech-stack/scripts/emit-cross-tool.sh

GATE: control plane grounded · mirrors emitted · quality lint green
```

CONTRACT	AGENT BEHAVIOR
Input	The signed-off task set and live repo. A technology with no in-scope task does not earn speculative harness surface.
Output	Grounded agents, KBs, doctrine, rules, closers, and mirrors. CONTEXT.md vocabulary is wired into the control plane.
On red	Repair missing coverage, ungrounded KBs, unwired closers, quality-lint failures, or cross-tool drift. Do not weaken the quality gate.
On green	Pass the exact task, cited decisions, and grounded harness to the execution loop. The current CLI does not yet orchestrate this stage.

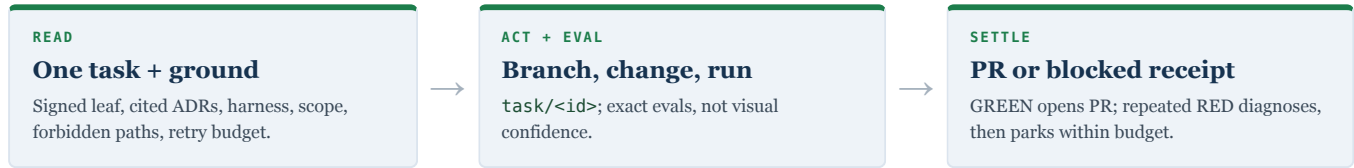
SHIPPED SKILL VS CLI SURFACE

The Harness stage exists as a skill and scripts; cvg 0.13.0 does not expose a harness command. v5 documents that boundary instead of drawing a command that has not shipped.

§ Agent flow · The Loop.

READ → ACT → EVAL → SETTLE · TASK-LOOP 0.3.0

The loop never chooses work; it receives exactly one ready issue. The execution agent reads the Task-Spec, its cited decisions, and the harness; works inside the declared paths; runs the task's own eval; and either opens a green-evidence PR or parks a bounded failure with context.



```

# current CLI can run the task's evals
cvg eval cvg/tasks/T-revenue-gold.md --json

# task-loop owns branch + bounded refinement + PR
bash skills/task-loop/scripts/run-issue-eval.sh --issue T-revenue-gold
RED → reproduce minimally → hypothesize → verify → fix → rerun
GREEN → PR with exact eval evidence → close issue

# post-execution acceptance
cvg tasks accept cvg/tasks/T-revenue-gold.md --stamp --gold-sanity
  
```

CONTRACT	AGENT BEHAVIOR
Input	One signed-off leaf selected by a human or scheduler. Missing spec, ADR, or harness is an upstream block, not permission to guess.
Output	A scoped branch and PR with green eval evidence, or a blocked-task report with the last output, hypothesis, and exhausted budget.
On repeated red	After the same failure twice, stop patching. Reproduce minimally, state a cause, verify it, then change once. Never consume the whole retry budget guessing.
Current boundary	No cvg run/work/board Manager exists yet. task-loop can execute one issue; scheduling, fan-out, CI merge gates, and board automation remain future machine work.

THE CLOSE CONDITION

Only the task's own green eval earns a PR and closes an issue. Passing means the implementation conforms to the encoded contract — not that the original business decision was necessarily correct.

Machine contracts — how the agent decides.

TOKENS · JSON ENVELOPE · EXIT TAXONOMY · MUTATION SEMANTICS

An autonomous caller needs a smaller, stricter surface than a human. Every cvg command therefore ends in a stable token and preserves a typed exit code. `--json` wraps the same command without changing the underlying verdict or byte-parity default path.

```
{
  "ok": true,
  "command": "decompose",
  "token": "CHECK_PLAN=OK",
  "verdict": "OK",
  "exit_code": 0,
  "changed": false,
  "dry_run": false,
  "data": { ... },
  "error": null,
  "warnings": [],
  "meta": { "schema_version": "1.0", "tool": "cvg", "cvg_version": "0.13.0" }
}
```

EXIT	MEANING	AGENT RESPONSE
0	OK	Persist receipt and advance. For a mutation, confirm changed matches expectation.
1	GATE_FAIL	Artifact is red. Repair at the current pass; a blind retry is not progress.
2	USAGE_ERROR	Bad flag, ambiguous discovery, or nothing to gate. Fix invocation or identify the artifact.
20	SKIP	Engine unavailable. Select an allowed ready engine or stop with an explicit coverage gap.
21	TIMEOUT	Retryable dispatch failure. Scope smaller or retry within policy; no fabricated artifact.
22	ENGINE_ERROR	Provider output could not become a valid judgment. Preserve raw output and diagnose adapter/format.

READ-ONLY BY DEFAULT

Gate without changing

capture, intent, structure, decompose,
checks, lint, ready, eval, manifest.

MUTATIONS ARE EXPLICIT

Preview, then act

adversary dispatch, acceptance stamps,
transitions, setup choice, registration. Use `--dry-run` where supported.

RECEIPTS, NOT STORIES

Persist the verdict

Store the token, exit, version, artifact
hash/path, and external ref. Human prose is
context, never the gate.

Evidence ledger — what exists, what is proved.

SNAPSHOT: FEAT/E2E @ EE53112 · 2026-07-23 · WORKING-TREE PRESENTATIONS EXCLUDED

A blueprint becomes misleading when it treats code-complete, fixture-proven, live-connected, and proving-ground-closed as the same state. v5 keeps those states explicit.

SURFACE	STATE	EVIDENCE BOUNDARY
Passes 0–4	CLOSED	Canonical proving-ground artifacts, direct-gate parity, and a real cross-family Consensus with provenance re-hash.
Tasking · Pass 5	ROUND ACTIVE	Nine backbone legs are nine specs; validate, DELEGATE, and DoD COMPLETE. The build contract still marks the wider R5 round in progress.
cvg core	SHIPPED LOCALLY	0.13.0: pass gates, agent-context, JSON, dry-run, DoD, setup, register. Branch is ahead of its recorded remote; no release tag beyond vo.1.
Task-Spec	3.5.0	Six-tier engine from 3.4 plus generic <code>tracker_ref</code> receipt and validator behavior in 3.5.
Register	IMPLEMENTED	Five-verb adapters + 46/46 fake-backend suite reported; live Linear setup found and fixed transport bug. Full real-board mapping closure is still pending.
Harness · Pass 6	SKILL EXISTS	Skill/scripts and gate contract exist; no <code>cvg harness</code> command and no current proving-ground close.
Loop · Pass 8	SINGLE ISSUE	<code>task-loop</code> executes one named issue; <code>cvg eval</code> runs evals. Manager, fan-out, CI gate, board, route/run, and scheduling remain planned.

IMPLEMENTED MEANS

The command or skill exists on disk and its documented contract can be inspected. It may also have fixture proof. This does not automatically close its method round.

CLOSED MEANS

The real pass ran on the proving ground, the artifact was canonized, the CLI reproduced the golden gate, the proof is durable, and the owner can be taught what closed.

PROVENANCE OF THIS DOCUMENT

Method and CLI claims were reconciled against `bin/cvg`, `bin/README.md`, the stage `SKILL.md` contracts, Task-Spec changelog, adapter contract, `cvg-todo.md`, and the proving-ground index. v5 does not treat the locally edited presentation snapshots as implementation proof.

Why this is defensible

TWO MOATS · POSITIONING · THE HONEST BOUNDARY

The two moats

Converge holds two moats, not one, and the discipline is to keep them separate.

OPTIONALITY

Task-Spec

the eval — not the agent — defines done, so you swap Codex / Kimi / Claude freely; the coding agent is a commodity slot.

QUALITY

Harness

the control-plane files that make any commodity agent succeed; not downloadable, built over thousands of hours.

TOGETHER

Defensible

run on commodity models, produce defensible output. Two moats — optionality and quality — not one.

Positioning vs the field

The canonical SDD pipeline is Specify → Plan → Tasks → Implement, punctuated by human checkpoints — Spec Kit, Kiro, OpenSpec, BMAD all share this shape, and their gates and loops keep growing. Converge's claim is not a feature checklist but a **difference in contract**: the adversarial **Consensus** pass, where a **different-family** model attacks each plan default-to-refuted; the **one-spine model** — one six-tier tasking engine that decomposes big work **in place** rather than routing out to a second paradigm; and the closed **Loop** of Pass 8, where every unit is born with a runnable eval and only a green eval closes an issue. The named SDD frameworks solve the coupled-spec case Converge now folds into a single **L leaf** — the bet is that recursive, eval-backed decomposition beats routing to another method.

THE HONEST BOUNDARY

The Loop converges to green evals, not to correct outcomes. A passing spec test guarantees only that the code matches the spec — if the spec is wrong, the code faithfully implements the wrong thing. The Dark Factory automates the build, not the judgment of whether the spec was right. The defense is structural and partial, not total: Pass 1's gate forces the spec to answer the brief, and Pass 4's adversary attacks it before any code is written. Stating this plainly is the point — the intellectual honesty is the moat's foundation, not a footnote.

The runtime, proven

WHAT THE SKILLS ALREADY ENFORCE — NOT A PROMISE, A MECHANISM

The gates above are not honour-system prompts. The task-spec skill ships the runtime that makes "done = proved" a mechanism a human cannot fake and an agent cannot shortcut. Five enforcement layers matter most, and all five already exist on disk.

LAYER	WHAT IT ENFORCES · WHERE IT LIVES
Atomic state machine	A locked, portable transition protocol — advisory lock (mkdir-based, macOS-safe), read current status, validate the transition against a real state machine, write via tmp-file + atomic move. Two agents cannot both claim one ready task. <code>transition-status.sh</code>
Tool-captured evidence	The acceptance gate re-runs the evals itself — it never trusts the executor's word. A non-zero exit is rejected automatically. This is the hole DGE's "evidence gate" fills; Converge fills it too. <code>accept-task.sh · GATE A</code>
Anti-reward-hacking	Beyond re-running: blast-radius check (changed files must be inside <code>touches_paths</code>), HMAC contract integrity (evals were not weakened after sign-off), and gold-sanity (the eval must FAIL on the unpatched baseline). <code>GATE B · C · E</code>
Rebuildable ledger	The T-spec frontmatter is truth; <code>_metrics.jsonl</code> is an append-only ledger; <code>_state.yaml</code> is a rebuildable index. The board is a projection of one source of truth — it cannot silently diverge. <code>rebuild-state.sh</code>
Teaching gate	The lesson is enforced, not hoped for: every artifact taught, every decision naming a rejected alternative, every term of art defined, 3–5 check-yourself questions, and a clean tree. A pass is not closed until its lesson exists. <code>check-lesson.sh</code>

Verified repository snapshot, 2026-07-23: Passes 0–4 closed end-to-end; the nine-leg backbone is fully tasked, validating, delegable, and DoD-complete; **cvg 0.13.0** and Task-Spec **3.5.0** are present locally. Register is implemented with offline adapter proof; Harness and Loop remain the next real closure rounds.

WHAT DOES NOT EXIST YET

The single-issue execution loop is not a fleet manager. Automated issue selection, fan-out, concurrency limits, CI-side re-verification, merge policy, board reconciliation, route/run commands, and portfolio metrics remain future machine work. A blueprint may show the destination; its evidence ledger must keep these surfaces labeled planned until they ship and pass.

WHAT THIS MEANS FOR AUTONOMY

Autonomy is the reward for understanding, not a substitute for it. Converge earns the right to run dark at Pass 8 by spending human judgment at Passes 1–4 — the swimlanes, the ADRs, the adversary. A running loop with no upstream comprehension builds the wrong thing flawlessly. The factory must run dark and be aimed correctly; these passes are what aim it.

One method, every engineer

THE ROLE LIVES IN THE EVAL, NOT THE LOOP

The loop never inspects what a task is — it inspects only one thing: does the task carry a runnable eval, and is that eval green. All role-specificity is pushed down into the per-task eval. The loop primitive — take an action, read the environment, decide, repeat until the goal condition holds — is identical whether the artifact is a SQL model, a service, an IaC module, or a retriever. That is precisely why Converge is canonical across disciplines: the role does not change the machine, it only changes the assertion the machine waits on.

ROLE	WHAT ITS TASK'S EVAL IS
Data Engineer	a dbt test + a control-sum query against gold
Software Engineer	a unit / integration test that passes
DevOps	terraform plan clean + a smoke test
AI Engineer	a retrieval-precision or eval-set threshold met

THE PRINCIPLE

The role lives in the eval, not the loop. task-loop runs whatever until its eval is green — and CI/CD schedules it — both role-agnostic by construction. That is why one method serves Data Engineering, Software, DevOps, and AI Engineering alike.

The method, end to end on one feature

POSTGRES → DUCKDB → DBT → MCP · TASK-DRIVEN

One feature carries the whole framework: **"daily revenue per product category, served via MCP so a non-engineer can ask for it."** The real metric is `SUM(payments.amount) WHERE payments.status='paid'`, grouped by `products.category` and `date(orders.ordered_at)` in UTC — every pass on the real repo. This feature is the proving ground itself — `tests/uc-analytics`, Postgres on 5433, deterministic seed 42.

PASS	WHAT HAPPENS ON THE REAL REPO
P0 · Capture	Raw idea → <code>docs/brd-analytical-backbone.md</code> in the owner's voice, signed 2026-07-19 — the first canonical artifact born on the proving ground. Gate: <i>check-brd.sh green</i> .
P1 · Intent	BRD → tech-spec: one gold model + one MCP tool over raw orders + payments + products. Gate: <i>the spec answers the brief</i> .
P2 · Structure	Brownfield. Open <code>src/db/01_schema.sql</code> ; write ADRs <code>0001-join-on-order-id</code> , <code>0002-utc-date-grain</code> , <code>0003-revenue-is-paid-only</code> . System understood vs reality.
P3 · Decompose	Two plans, plan-altitude only: Plan A transform (<code>silver</code> → <code>gold_daily_revenue</code>), Plan B serve (<code>mcp tool query_daily_revenue</code>).
P4 · Consensus	<code>cvg review --adversary codex attacks: "midnight off-by-one (no TZ)? refunds counted?"</code> Fix: pin UTC (ADR-0002). Accept: refunds out of v1. Hands directly to Tasking.
P5 · Tasking	<code>tasks/build_gold_revenue</code> (eval: <code>dbt test + control-sum</code>) and <code>tasks/build_mcp_revenue</code> (eval: <code>tool result == gold query</code>). Each born with its eval.
① Register	Operational target: <code>cvg register --tracker linear projects build_gold_revenue</code> → ISSUE-41 [ready] and <code>build_mcp_revenue</code> → ISSUE-42 [blocked-by 41], then stamps generic receipts. The bridge exists; this exact live mapping is not presented as completed proof.
P6 · Harness	Next closure: <code>stack = dbt + duckdb + mcp</code> → <code>scaffold .claude/</code> with paired agents, KBs, closers, doctrine, and <code>AGENTS.md</code> . The skill exists; the proving-ground run remains ahead.
P8 · The Loop	Target behavior: one named issue → scoped branch → exact eval RED → diagnosis → GREEN → evidence PR. The single-issue skill exists; the unattended Manager/fleet does not.

THE DARK FACTORY TARGET, ON THIS REPO

A non-engineer asks the MCP *"revenue by category yesterday?"* — and it answers. Brief → deployed capability, every step eval-verified, the human holding only the judgment gates. v5 shows the complete target while the evidence ledger makes clear that Harness, live Register closure, and the unattended Loop still have to earn that ending.